운영 중 변경되는 보상을 안전하게 지급한 독서 보상 API

ABC(Acorn Book Cloud) · Backend Portfolio Case Study
손영석 · Java / Spring Boot 백엔드 개발자

30초 요약

ABC는 전자책 대여·열람과 독서 활동 보상을 제공한 5인 팀 프로젝트다. 부팀장과 백엔드 개발을 맡아, 대표적으로 보상 지급 과정의 **정책 변경, 중복 수령, 부분 반영** 문제를 해결했다.

아직 수령하지 않은 행만 변경하는 조건부 갱신으로 재지급을 막고, 수령 상태와 포인트·쿠폰 지급을 하나의 트랜잭션으로 처리했다. 성공·반복·실패 시나리오는 API 응답과 최종 DB 상태를 대조해 검증했다.

구분	내용
프로젝트 기간	2026.06.12 ~ 2026.07.27(발표)
팀·역할	5인 팀 부팀장 · DB 설계 참여 · 백엔드 개발 · 발표
대표 담당	챌린지·출석·포인트·쿠폰 백엔드 API와 관련 DB·검증
핵심 기술	Java 17 · Spring Boot 4.0.7 · Spring Data JPA · MySQL · REST API

핵심 결과

검증 항목	확인 결과
중복 수령 방어	빠른 반복 요청에서 첫 요청 `200`, 다음 요청 `409`
포인트 지급 정합성	잔액 `300 → 400`, 포인트 이력 1건
쿠폰 실제 지급	응답 수량 2개, `member_coupon` 2건
실패 시 롤백	`reward_claimed_at=NULL`, 신규 지급 이력 없음

1. 문제 정의 — 달성과 지급 사이에는 시간이 존재한다

사용자가 챌린지를 달성했다고 해서 지급까지 끝난 것은 아니다. 보상 수령 전까지 관리자가 보상 설정을 변경할 수 있고, 사용자의 버튼 반복 클릭이나 네트워크 재시도로 같은 요청이 다시 들어올 수도 있다.

핵심 질문	방치했을 때 생기는 문제
달성 후 보상이 바뀌면 어느 값을 지급할 것인가?	사용자별 지급 기준이 달라지고 운영 정책을 설명하기 어려움
같은 보상 수령 요청이 반복되면 누가 지급받는가?	포인트 또는 쿠폰 중복 지급
수령 완료를 기록한 뒤 실제 지급이 실패하면 어떻게 되는가?	보상은 받지 못했는데 재수령도 할 수 없는 부분 반영

이 사례의 목표는 사용자의 보상 수령 한 번을 신뢰할 수 있는 하나의 데이터 변경으로 만들고, 재시도하더라도 같은 보상이 중복 지급되지 않게 하는 것이었다.

2. 내가 맡은 구현과 개발 방식

대표 사례에서 나는 출석 처리, 챌린지 진행도 갱신, 보상 수령, 포인트 이력과 회원 쿠폰 생성 로직을 구현하고 API·DB 결과를 검증했다.

```
사용자가 출석한다(U-016)
  → 당일 중복 출석 차단
  → 출석 기반 챌린지 진행도 갱신
  → 달성한 챌린지의 보상 수령을 요청한다(U-027)
  → 달성·미수령 상태 확인
  → 조건부 갱신으로 한 요청만 수령권 선점
  ├ 포인트: 회원 잔액 변경 + 포인트 이력 생성
  └ 쿠폰: 지급 수량만큼 회원 쿠폰 생성
  → 전체 성공 시 커밋, 하나라도 실패하면 롤백
  → 포인트·쿠폰 내역에서 실제 지급 결과 조회(U-017)
```

개발 과정은 다음 순서로 진행했다.

```
API 명세·ERD 확인
  → 구현 계획
  → Spring Boot·JPA 구현
  → PR 코드리뷰 대응
  → Swagger·Postman 검증
  → MySQL 사전·사후 결과 비교
```

구분	기여 범위
본인	보상 API·관련 DB 흐름·정책·트랜잭션 구현, 자동·수동 검증
추가 구현·보완	도서·카테고리·컬렉션·전자책·관리자 API 일부
본인 프런트 기여	기존 React 화면과 API wrapper·응답 타입·상태 갱신·오류 처리 연결
팀원	프런트 화면 디자인·페이지·라우팅 구현, 배포 구성
구현 근거	Backend #33·#35·#36, Frontend #100의 문제·판단·수정·검증 기록

따라서 신규 화면이나 신규 라우트를 개발한 것으로 표현하지 않고, 기존 화면과 백엔드 API를 연결한 범위만 개인 기여로 구분한다.

구현 범위에서도 다음 항목은 제외했다.

- 7일·30일 연속 출석 자체에서 쿠폰을 직접 지급하는 기능
- 달성 시점 보상 스냅샷 저장
- 관리자 달성 조건 변경 후 과거 진행도의 소급 재계산
- 실제 고부하 환경의 동시 요청 검증

3. 핵심 판단 1 — 보상은 언제 확정할 것인가

Backend PR #33 코드리뷰에서 "이미 달성했지만 아직 수령하지 않은 사용자가 있을 때 관리자가 보상을 바꾸면 어떤 값을 지급해야 하는가?"라는 문제가 제기됐다.

```
월요일: 사용자 A가 챌린지 달성 / 보상 100포인트 / 미수령
화요일: 관리자가 같은 챌린지 보상을 200포인트로 변경
수요일: 사용자 A가 보상 수령 요청
```

선택지	사용자 A 지급값	장점	비용·위험
달성 시점 스냅샷	100포인트	달성 당시 약속 보존	사용자별 보상 버전과 스냅샷 구조 필요
수령 시점 최신값	200포인트	운영 중 잘못된 설정 수정 가능, 별도 스냅샷 불필요	수령 전 보상값이 바뀔 수 있음

프로젝트에서는 **수령 시점의 최신 보상을 지급하는 정책**을 선택했다.

- 달성했지만 미수령인 사용자는 수령할 때 최신 `challenge_reward`를 조회한다.
- 관리자가 보상을 변경해도 이미 달성한 진행 상태는 초기화하지 않는다.
- 서비스 코드 주석과 `ChallengeServiceTest` 회귀 시나리오로 정책을 고정했다(E-05).

이 선택은 제한된 일정에서 별도 스냅샷 구조를 추가하지 않고 운영자가 잘못된 설정을 수정할 수 있게 한 결정이다. 실제 서비스라면 보상 변경 알림이나 달성 시점 스냅샷을 함께 검토해야 한다.

관리자가 **달성 조건**을 변경했을 때 과거 사용자의 진행도를 어떻게 처리할지는 이번 사례에서 해결했다고 주장하지 않는다. 확인된 정책은 수령 시점의 보상 종류·수량에 관한 결정이다.

4. 핵심 판단 2 — 같은 보상은 한 요청만 선점한다

조회 후 지급하는 구조에서는 두 요청이 모두 "미수령"을 읽을 수 있다.

```
요청 A: 미수령 조회 ┐
                    ├ 둘 다 지급으로 진행할 위험
요청 B: 미수령 조회 ┘
```

이를 막기 위해 Spring Data JPA 조건부 `UPDATE`에 `reward_claimed_at IS NULL`을 포함하고, 영향받은 행이 `1`일 때만 지급하도록 했다.

> **조건부 UPDATE:** 아직 수령하지 않은 행만 한 번 변경하는 DB 명령이다. 다른 요청이 먼저 처리했다면 변경되는 행이 없으므로 지급 단계로 넘어가지 않는다.

```
UPDATE challenge_progress
   SET reward_claimed_at = 현재 시각
 WHERE challenge_progress_id = ?
   AND reward_claimed_at IS NULL

영향 행 1 → 선점 성공, 지급 진행
영향 행 0 → 다른 요청이 이미 선점, 409 응답
```

이 방식은 별도의 조회 결과를 믿는 대신 DB가 실제로 변경한 행 수를 수령권 판단 기준으로 사용한다.

검증 범위도 구분했다.

- Repository 테스트: 같은 진행도에 대한 첫 조건부 갱신 `1`, 다음 갱신 `0`
- Postman Runner 빠른 반복: 첫 요청 `200`, 다음 요청 `409`(E-03)

5. 핵심 판단 3 — 수령 상태와 실제 지급의 불일치를 트랜잭션으로 방지한다

수령권을 먼저 선점하더라도 지급에 실패하면 사용자는 보상을 받지 못하면서 재수령도 할 수 없다. 따라서 수령 시각 기록과 실제 지급 데이터를 하나의 DB 트랜잭션으로 묶었다.

> **트랜잭션:** 관련된 DB 작업이 모두 성공하면 반영하고, 하나라도 실패하면 모두 취소하도록 묶는 단위다.
>
> **불변 조건: 지급에 실패하면 수령 완료 상태도 남기지 않는다.**

```
challenge_progress.reward_claimed_at 조건부 기록
  ├─ 포인트: member.point_balance 변경 + point_history 생성
  └─ 쿠폰: rewardQuantity만큼 member_coupon 생성
  → 모두 성공: COMMIT
  → 비즈니스 예외 또는 DB 실패: ROLLBACK
```

보상	함께 변경하는 데이터	일관성 기준
포인트	회원 잔액, `point_history`	잔액 변화량과 이력 금액이 같아야 함(E-01)
쿠폰	수령 시각, 수량만큼의 `member_coupon`	응답 수량과 생성 행 수가 같아야 함(E-02)
지원하지 않는 유형	지급하지 않음	예외 후 수령 시각과 새 지급 이력이 남지 않아야 함(E-04)

포인트 차감에도 잔액이 음수가 되지 않는 조건부 갱신을 사용하고, 성공한 경우에만 이력을 남겼다. 쿠폰은 `rewardQuantity`만큼 회원 쿠폰을 생성하고, 발급 시각과 유효기간으로 만료 시각을 계산해 `grantedCouponQuantity`로 실제 지급 수량을 반환했다.

트랜잭션이 보장하는 범위는 서버의 DB 작업까지다. 커밋 후 HTTP 응답 전달이나 프런트 화면 렌더링 실패까지 DB 트랜잭션이 되돌린다고 설명하지 않는다.

6. 기술·데이터 구조·API 계약

핵심 데이터 관계

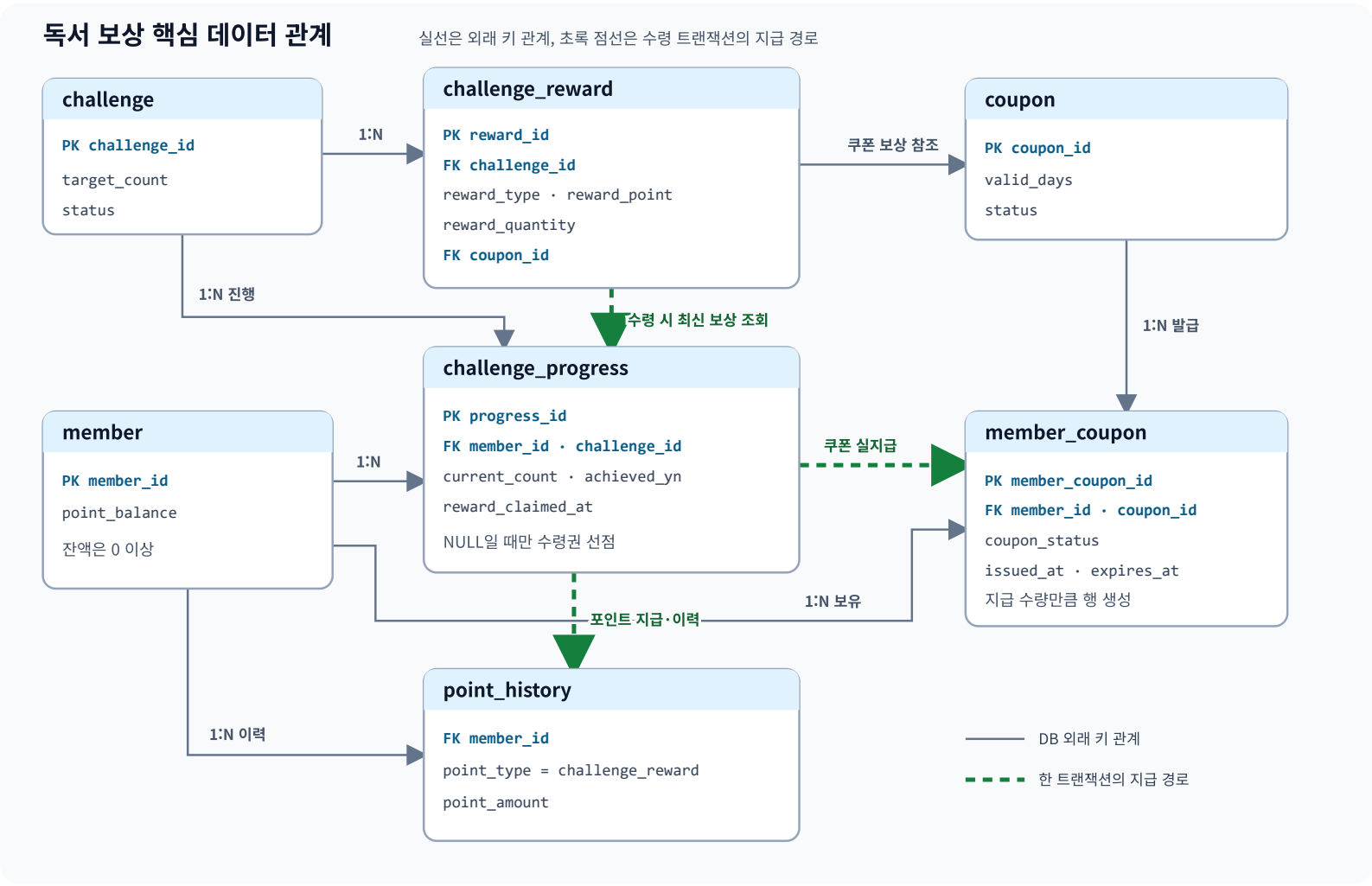


그림 1. 보상 지급에 직접 관여하는 테이블과 핵심 키·컬럼. `challenge_progress.reward_claimed_at` 선점 후 포인트 또는 회원 쿠폰 지급까지 같은 트랜잭션에서 처리한다.

기술을 선택한 이유

기술	이 사례에서 맡은 역할	확인한 결과
Spring Boot 서비스·트랜잭션	지급 절차 조율, 예외 처리, 선점과 지급의 트랜잭션 경계 설정	지급 예외 후 수령 시각과 지급 이력 미반영
Spring Data JPA	조건부 `UPDATE` 실행, 영향 행 수 반환, 포인트 이력·회원 쿠폰 저장	영향 행 수 `1/0`으로 선점 성공 여부 판정
MySQL	조건부 `UPDATE`를 원자적으로 실행하고 `CHECK`·`UNIQUE` 제약으로 스키마 불변 조건 보조	중복 수령 판단에 필요한 단일 갱신과 잘못된 데이터 상태 방어
REST API	정상·중복·실패를 명시적 계약으로 구분	`200`, `409`, 도메인 오류 코드
Swagger·Postman	정상·실패·반복 요청 재현	API 응답과 DB 결과를 분리해 검증

중복 수령을 직접 차단하는 기준은 MySQL 제약조건 하나가 아니라 `reward_claimed_at IS NULL` 조건부 갱신과 영향 행 수 판정이다. 제약조건은 상태값·관계 등 스키마 수준의 불변 조건을 보조하고, Spring 트랜잭션은 선점 이후 실제 지급까지 모두 성공하거나 모두 롤백되도록 책임을 나눴다.

DB 설계에 참여했으며, 이 사례에서는 `challenge_reward`·`challenge_progress`·`point_history`·`member_coupon` 관계와 제약을 지급 흐름에 연결하고 JPA 매핑 및 사전·사후 데이터 검증에 사용했다.

대표 API 계약 — API-CHALLENGE-002

항목	계약
목적	달성했지만 아직 수령하지 않은 챌린지 보상 지급
Method·Path	`POST /api/v1/challenges/{challengeId}/reward`
인증	JWT로 인증된 로그인 회원
요청 값	Path Variable `challengeId`
성공	`200 OK` · `rewardClaimedAt`, `grantedPoint`, `grantedCouponQuantity` 반환
중복·수령 불가	`409 Conflict` · `CHALLENGE_REWARD_NOT_CLAIMABLE`
미지원 보상 유형	`CHALLENGE_REWARD_TYPE_NOT_SUPPORTED` 반환 후 수령 상태와 지급 데이터 롤백
DB 결과	수령 시각 기록 + 포인트 잔액·이력 또는 지급 수량만큼 회원 쿠폰 생성

이 계약은 별도 API 명세 파일을 열지 않아도 요청, 응답, 오류, 최종 DB 결과를 한 번에 확인할 수 있도록 본문에 직접 수록했다.

API 계약은 구현의 오류 코드와 연결하고, 데이터 관계는 JPA 매핑과 DB 사전·사후 비교로 검증했다. 중복 수령과 부분 반영 방어 결과는 E-01~E-04에서 확인할 수 있다.

7. 검증 — 응답이 아닌 최종 데이터를 확인한다

문서 안에서 바로 확인할 수 있는 증거

증거	확인 방법	결과
E-01 포인트 지급	API 응답과 DB 사전·사후 비교	`grantedPoint=100`, 잔액 `300 → 400`, `point_history` 1건
E-02 쿠폰 지급	API 응답 수량과 회원 쿠폰 행 비교	`grantedCouponQuantity=2`, `member_coupon` 2건
E-03 중복 방어	Postman으로 같은 보상을 빠르게 두 번 요청	첫 요청 `200`, 반복 요청 `409`
E-04 실패 롤백	미지원 보상 요청 후 DB 조회	`reward_claimed_at=NULL`, 신규 지급 이력 없음
E-05 회귀 검증 기록	Backend #33·#35·#36 PR 본문의 테스트 대상·결과 요약	정책 변경, 조건부 갱신, 실패 롤백, 쿠폰 실지급 시나리오 확인

요청 응답과 DB 조회 결과는 같은 검증 시점에 저장한 자료를 사용했다. JSON 전체와 DB 원문 대신 핵심 필드와 행을 비전공자도 읽을 수 있는 설명과 표로 부록 B에 재구성했다.

자동 검증 범위

- `ChallengeServiceTest`: 수령 시점 최신 보상, 미지원 유형 실패, 쿠폰 실제 지급
- `ChallengeProgressRepositoryTest`: 첫 조건부 갱신 `1`, 다음 갱신 `0`
- `PointServiceTest`: 조건부 잔액 변경, 성공 시 이력 저장, 잔액 부족 시 이력 미생성

E-05는 작성 당시 PR 본문에 남긴 테스트 대상과 실행 결과의 요약이다. 별도의 테스트 실행 화면은 확보하지 않았으므로, 고정된 전체 통과 건수나 테스트 파일 전체의 단독 소유를 주장하지 않는다.

8. 코드리뷰와 Git 협업

기능 개수보다 리뷰에서 발견한 위험을 정책과 회귀 테스트로 바꾼 과정에 집중했다. 외부 링크 없이도 맥락을 알 수 있도록 PR 번호와 핵심 내용을 함께 적었다.

Backend #33 — 보상 정책과 중복 수령

- 문제: 달성 후 아직 받지 않은 보상이 변경될 때 적용 기준이 불명확했고, 조회 후 갱신 방식에는 중복 수령 위험이 있었다.
- 판단·수정: 수령 시점의 최신 보상을 적용하는 정책을 명문화하고, 조건부 갱신의 영향 행 수로 한 요청만 수령권을 얻도록 했다.
- 검증: `ChallengeServiceTest`, `PointServiceTest` 관련 회귀 시나리오와 빠른 반복 요청 `200 → 409`를 확인했다.

Backend #35 — 실패 처리와 롤백

- 문제: 지원하지 않는 보상 유형도 지급 단계까지 진행되어 수령 상태만 남을 수 있었다.
- 판단·수정: 미지원 유형을 지급 전에 차단하고, 지급 중 예외가 발생하면 수령 시각을 포함한 전체 DB 작업이 취소되도록 했다.
- 검증: `ChallengeProgressRepositoryTest`에서 조건부 갱신 결과 `1 → 0`을 확인하고, 실패 후 `reward_claimed_at=NULL`을 대조했다.

Backend #36 — 쿠폰 실지급

- 문제: 보상 수령 완료 상태와 실제 회원 쿠폰 생성 결과가 달라질 수 있었다.
- 판단·수정: 설정된 수량만큼 `MemberCoupon`을 생성하고, 실제 생성된 수량을 응답하도록 했다.
- 검증: 응답의 쿠폰 수량과 생성된 `member_coupon` 행 수가 모두 2개인지 확인했다.

Frontend #100 — A-013 포인트 연동

- 문제: 임의 잔액을 사용하고 실패한 요청도 화면에서 성공처럼 처리할 수 있었다.
- 판단·수정: 실제 잔액 조회 API를 연결하고, 서버가 성공을 응답한 경우에만 화면 상태를 갱신하도록 했다.
- 검증: 수정 파일의 TypeScript·ESLint 정적 검증을 수행했고, 현재 팀 배포 화면에서 실제 잔액 연동 결과를 확인했다. PR 당시 브라우저 수동 검증을 수행했다는 주장은 하지 않는다.

9. 팀 배포 환경에서 확인한 서비스 결과

팀 배포 환경에서 내가 구현한 API의 연동 결과를 확인했다. 배포 구성은 팀원이 담당했으며, 아래 화면은 API·DB 검증을 보완하는 서비스 결과 자료다. 외부 접근이 필요한 배포 URL이나 저장소 주소는 구현 근거로 사용하지 않았다.

직접 연동한 관리자 포인트 화면

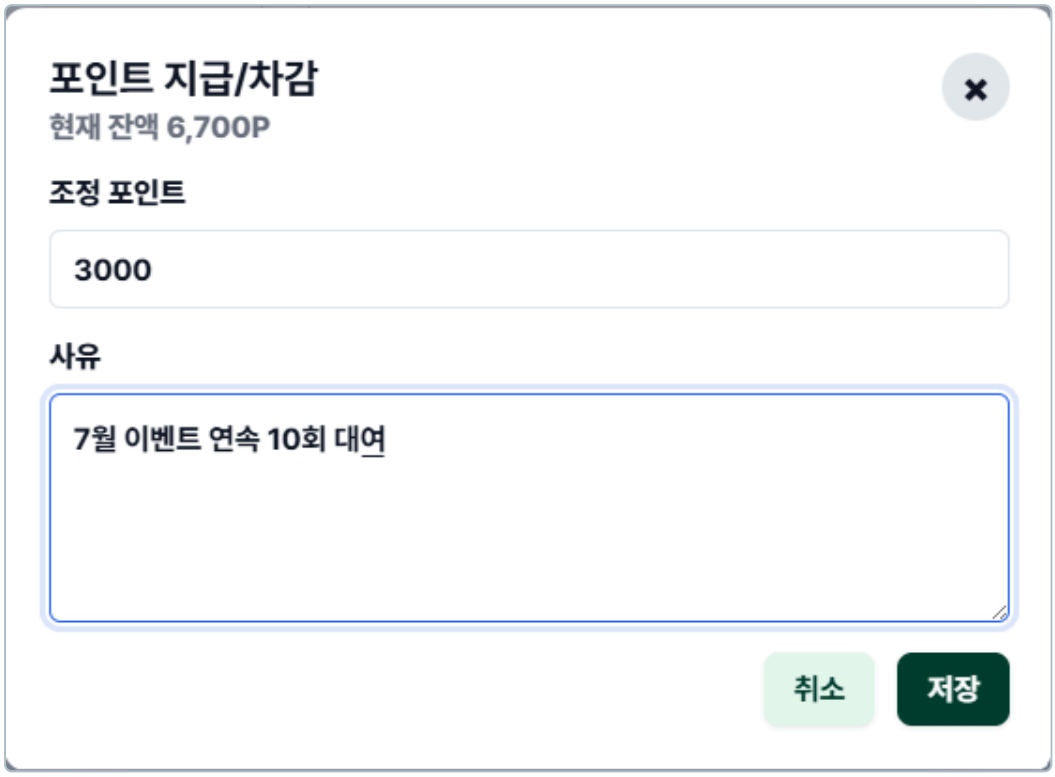


그림 2. A-013 관리자 포인트 조정 모달. 팀원이 구현한 화면에 실제 잔액 조회와 지급·차감 API를 연결했다. 이 화면은 연동 대상을 보여 주며, 지급 결과의 직접 근거는 E-01의 API 응답과 DB 이력이다.

백엔드 API 결과를 보여 주는 팀 화면

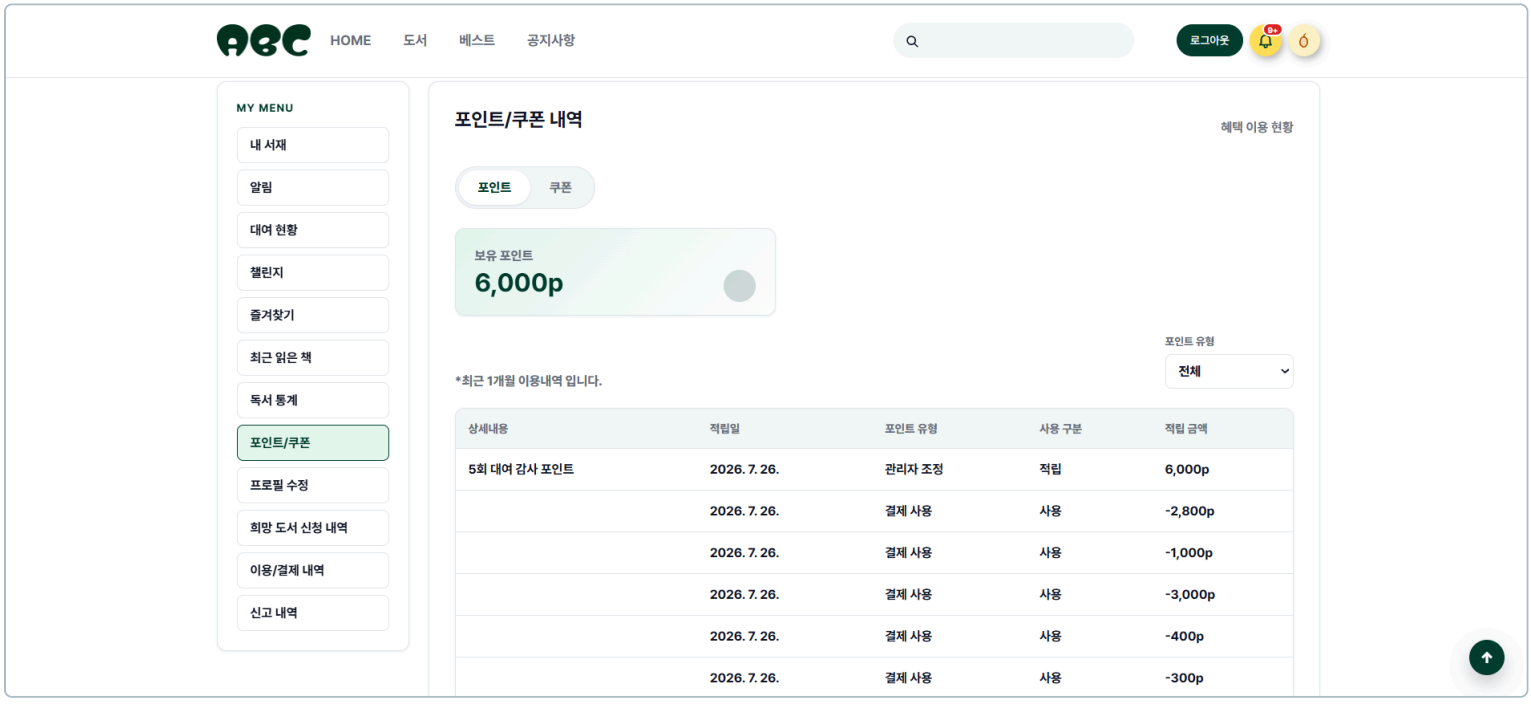


그림 3. 팀원이 구현한 사용자 화면에서 내가 개발한 포인트 조회 API의 잔액과 적립·사용 이력을 표시한 결과. 특정 챌린지 지급 건의 직접 근거는 E-01의 API·DB 자료다.

배포 환경에는 문서용 챌린지 데이터를 추가할 운영 권한이 없어 챌린지 화면은 제외했다.

10. 결과·한계·회고

개선 전후

개선 전 위험	적용한 해결	확인 결과
조회 결과에 의존하면 여러 요청이 모두 미수령으로 판단할 수 있음	조건부 갱신의 영향 행 수로 수령권 판정	첫 요청 `200`, 반복 요청 `409`(E-03)
수령 완료와 포인트 지급·이력이 따로 반영될 수 있음	선점·잔액 변경·이력 생성을 한 트랜잭션으로 처리	잔액 `300 → 400`, 이력 1건(E-01)
쿠폰 보상이 완료 상태만 남길 수 있음	지급 수량만큼 회원 쿠폰을 실제 생성	응답 2개, DB 2건(E-02)
지급 실패 후 수령 완료 상태가 남을 수 있음	비즈니스 예외 발생 시 전체 롤백	수령 시각 `NULL`, 신규 이력 없음(E-04)
화면이 서버 실패와 무관하게 성공을 표시할 수 있음	서버 성공 응답을 화면 상태의 기준으로 사용	실제 잔액과 서버 결과를 반영한 A-013 연동

검증·기여 범위

- E-03은 같은 보상을 빠르게 반복 요청한 수동 확인이며, 실제 동시 부하 테스트는 아니다. 추후 Testcontainers와 동시 요청 통합 테스트로 경합 상황을 재현할 수 있다.
- DB 설계에 참여하고 보상 관련 테이블 관계를 구현·검증했지만, 프로젝트 전체 스키마를 혼자 설계했다고 주장하지 않는다.
- E-05는 PR 본문에 기록된 관련 테스트 범위의 요약이다. 별도의 실행 화면이나 전체 테스트의 고정 통과 건수는 제시하지 않는다.

현재 한계와 개선 방향

현재 한계	개선 방향
수령 시점 최신 보상 정책이 변경 사실을 사용자에게 알리지 않음	보상 버전·달성 시점 스냅샷 또는 변경 알림 도입
7일·30일 출석에서 쿠폰을 직접 지급하지 않음	출석 보상 쿠폰 식별·중복 지급 기준 확정
쿠폰 활성 상태·만료 전환 정책이 완전하지 않음	`ACTIVE` 검증과 만료 쿠폰 상태 동기화를 발급·사용 공통 정책으로 적용

처음에는 출석, 챌린지, 포인트, 쿠폰을 각각의 CRUD로 보았다. 코드리뷰를 거치며 보상 기능의 핵심은 API 개수가 아니라 **시간에 따라 바뀌는 상태와 운영 정책을 일관되게 처리하는 것**임을 배웠다. 이후에는 성공 응답만 확인하지 않고 잔액, 이력, 회원 쿠폰까지 대조해 서버가 만든 최종 상태를 검증했다.

부록 A. 대표 API와 추가 기여 범위

API ID	Method	대표 사례에서의 역할
API-ATTEND-001	`POST /api/v1/me/attendance`	출석 처리와 챌린지 진행도 갱신
API-POINT-001	`GET /api/v1/me/points`	지급 후 포인트 잔액·이력 조회
API-COUPON-001	`GET /api/v1/me/coupons`	지급 후 회원 쿠폰 조회
API-CHALLENGE-001	`GET /api/v1/challenges`	챌린지 진행·달성·수령 상태 조회
API-CHALLENGE-002	`POST /api/v1/challenges/{challengeId}/reward`	대표 보상 수령 API
API-ADMIN-COUPON-003	`POST /api/v1/admin/coupons/{couponId}/issue`	관리자의 회원 쿠폰 발급
API-ADMIN-MEMBER-004	`POST /api/v1/admin/members/{memberId}/points`	A-013 관리자 포인트 조정 연동

그 밖에 도서·카테고리·컬렉션·전자책·관리자 대시보드·통계·감사 로그 API 일부를 구현·보완했다. Frontend #99에서는 기존 사용자 컬렉션과 관리자 도서·컬렉션·회원 화면을 백엔드 계약에 연결했다. 대표 보상 사례와 직접 연결되지 않는 기능은 본문에서 제외했다.

부록 B. API·DB 검증 자료

B-1. E-01 — 포인트 보상


```
{
  "data": {
    "rewardClaimedAt": "2026-07-07T21:38:46.6675009",
    "grantedPoint": 100
  },
  "success": true
}
```


- `grantedPoint=100`: 이번 요청에서 실제 지급한 포인트
- `rewardClaimedAt`: 해당 보상을 수령 완료로 기록한 시각
- `success=true`: 서버의 지급 처리가 정상 종료됐다는 의미

DB 확인 항목	저장 결과
회원 포인트	`300 → 400`
챌린지 진행 상태	`achieved_yn=1`, `reward_claimed_at=2026-07-07 21:38:47`
포인트 이력	`point_type=challenge_reward`, `point_amount=100`, 1건

E-01 판정: API가 100포인트 지급을 응답했고, 같은 검증 시점의 DB에서 회원 잔액 증가분과 포인트 이력 금액이 모두 100으로 일치했다.

B-2. E-02 — 쿠폰 보상


```
{
  "data": {
    "rewardClaimedAt": "2026-07-10T10:17:02.2802439",
    "grantedPoint": 0,
    "grantedCouponQuantity": 2
  },
  "success": true
}
```


- `grantedCouponQuantity=2`: 회원에게 생성된 쿠폰 수량
- `grantedPoint=0`: 쿠폰 보상에서는 포인트를 함께 지급하지 않음

발급된 회원 쿠폰	상태	발급 시각	만료 시각
1건째	`ISSUED`	2026-07-10 10:17:02	2026-07-17 10:17:02
2건째	`ISSUED`	2026-07-10 10:17:02	2026-07-17 10:17:02

E-02 판정: API가 쿠폰 2개 지급을 응답했고, 같은 검증 시점의 DB에서 서로 다른 회원 쿠폰 행 2건이 생성됐다.

B-3. E-03 — 빠른 반복 수령

요청	결과	의미
첫 수령	`200`, `grantedPoint=100`	지급 성공
반복 수령	`409`, `CHALLENGE_REWARD_NOT_CLAIMABLE`	이미 수령한 보상의 재지급 차단

E-03 판정: Postman으로 동일 보상을 빠르게 두 번 요청했을 때 첫 요청만 성공하고 다음 요청은 수령 불가 오류로 종료됐다. 이는 수동 반복 요청 결과이며 assertion 자동 테스트나 실제 동시 부하 테스트 결과로 확대 해석하지 않는다.

B-4. E-04 — 지급 실패

```
{
  "code": "CHALLENGE_REWARD_TYPE_NOT_SUPPORTED",
  "message": "아직 지원하지 않는 보상 유형입니다.",
  "success": false
}
```

실패 후 DB 확인 항목	결과
챌린지 달성 상태	`achieved_yn=1` 유지
보상 수령 시각	`reward_claimed_at=NULL`
새 챌린지 보상 이력	생성되지 않음

E-04 판정: 지원하지 않는 보상 유형에서 오류가 발생한 뒤에도 달성 상태만 유지됐고, 수령 시각과 신규 지급 이력은 남지 않았다.